

ubot: Master Setup and Work Log

What this is. This document records everything done to the ubot workspace between 29 September and 1 October 2026. It has four uses:

- rebuilding the whole setup from an empty machine (sections 1–4);
- running the benchmark (section 5);
- knowing what was changed and why (section 6);
- knowing **how each component was created from scratch**: commands, derivations, algorithms and the checks behind every finding (section 7, with the `diag` tests in 7.14).

Resuming after a shutdown? Go straight to **section 5.1**. The pilot was stopped on 1 Oct 2026 at 08:50 with 2 of 24 trials done, and continues from where it stopped.

[PROTOCOL.md](#) is the research design (what is measured and how it is analysed). This document is how to reproduce the environment and the work. The two are kept in sync.

Repository: Unipod-Robotics/uni-bot (GitHub). Branch `feat/nav-benchmark`, based on `imu-bno085-ekf`. Commits are listed in section 7.

1. Reference machine

The setup was built and tested on this machine. Other machines should match the bold items.

Item	Version
OS	Ubuntu 24.04 LTS (noble)
ROS 2	Jazzy (rclpy 7.1.11)
Gazebo	Harmonic (gz-harmonic 1.0.0, gz-sim 8, gz-physics 7 with DART)
gz_ros2_control	1.2.17
Nav2 / nav2_bringup	1.3.11
slam_toolbox	2.8.4 (a lifecycle node on Jazzy)
robot_localization	3.8.3
GPU	NVIDIA GeForce MX570 A, driver 580.159.03 (headless Gazebo rendering uses NVIDIA EGL)
CPU / RAM	12 cores / 31 GB
Python analysis	venv <code>~/uni-bot/.venv-bench</code> : evo 1.31.1, statsmodels 0.15.0, pandas 3.0.3, markdown 3.11

2. Fresh-machine setup

2.1 System packages

```
# ROS 2 Jazzy desktop and Gazebo Harmonic (ros-jazzy-ros-gz) are assumed installed.
sudo apt install ros-jazzy-nav2-bringup ros-jazzy-slam-toolbox ros-jazzy-robot-localization \
ros-jazzy-gz-ros2-control ros-jazzy-ros2-controllers ros-jazzy-twist-stamper \
ros-jazzy-depthimage-to-laserscan ros-jazzy-rosbag2-storage-mcap ros-jazzy-xacro \
python3-vcstool python3-scipy python3-psutil python3-venv
# Needed by the real-robot bringup and ubot_mono_nav (not installed on the reference machine
```

```
yet):
sudo apt install ros-jazzy-tf-transformations python3-smbus
```

2.2 Workspace

```
git clone git@github.com:Unipod-Robotics/uni-bot.git ~/uni-bot      # the repo root IS the
workspace
cd ~/uni-bot && git checkout feat/nav-benchmark
vcs import --skip-existing src < src/ubot/ubot.repos              # third-party drivers (git-
ignored)
src/ubot/ubot_worlds/scripts/fetch_models.sh                     # AWS world models, pinned
commit
```

- Cloning over HTTPS needs credentials; SSH was used for pushes.
- `fetch_models.sh` downloads ~140 MB from `zp78-ship-it/turtlebot-maze` at commit `30424d5` into `src/ubot/ubot_worlds/models_external/`, which is git-ignored.

2.3 Build

```
conda deactivate          # an active conda env makes CMake pick the wrong Python
source /opt/ros/jazzy/setup.bash
cd ~/uni-bot
colcon build --symlink-install --cmake-args -DBUILD_TESTING=OFF
source install/setup.bash
```

- `-DBUILD_TESTING=OFF` is required: `topic_based_ros2_control`'s tests need `ros_testing`, which is not installed.
- `ubot_worlds` and `ubot_description` install **copies** of their data files. Rebuild them after editing a world, mission, map or URDF. `bench run` refuses to start if `ubot_worlds` is stale.

2.4 Analysis environment

```
python3 -m venv --system-site-packages ~/uni-bot/.venv-bench      # keeps rclpy importable
~/uni-bot/.venv-bench/bin/pip install -r src/ubot/ubot_bench/requirements-bench.txt markdown
```

2.5 Headless rendering

Gazebo sensors need a GPU context even headless. The orchestrator sets this automatically; for manual runs:

```
export __EGL_VENDOR_LIBRARY_FILENAMES=/usr/share/glvnd/egl_vendor.d/10_nvidia.json
```

3. Check that the setup works

Check	Command	Expected
Interactive sim	<code>ros2 launch ubot_bringup sim.launch.py</code> (add <code>world:=arena_5x5</code> , <code>sensor_profile:=lidar_ld06</code> as needed)	Gazebo shows the small house, the robot spawns, RViz shows /scan and the IMU axes on the robot
Teleop	<code>ros2 run teleop_twist_keyboard</code> <code>teleop_twist_keyboard</code>	Drives in Gazebo with no sideways creep
Worlds render	<code>ros2 launch ubot_worlds</code> <code>gz_world.launch.py world:=bookstore</code> <code>condition:=glass</code>	World loads with no Ogre crash

Check	Command	Expected
Smoke experiment	<code>ros2 run ubot_bench bench run smoke</code>	4 trials (REF + OAKD, arena, both phases) ok
Analysis	<code>~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.report smoke</code>	bench_results/smoke/analysis/ holds CSVs, figures and tables

4. Regenerating the committed data (only if worlds change)

Ground-truth maps and missions are committed. They are regenerated only when a world or the generation code changes, and always in this order:

```
ros2 run ubot_bench gt_map --world arena_5x5          # ~1 min
ros2 run ubot_bench gt_map --world small_house        # ~20 min under load
ros2 run ubot_bench gt_map --world bookstore          # ~15 min
ros2 run ubot_bench gt_map --world small_warehouse --lattice 0.75
for w in arena_5x5 small_house bookstore small_warehouse; do ros2 run ubot_bench make_missions
--world $w; done
colcon build --packages-select ubot_worlds             # install the new missions and maps
```

- Inspect `ubot_worlds/missions/<world>.png`. Goals and the route must be inside the building.
- The AWS world xacros are generated by `ubot_worlds/scripts/import_aws_world.py <src> <out> <name>` from the pinned `tb_worlds/worlds/*.sdf.xacro`. Re-run it only if the import logic changes.

5. Running the benchmark

Experiment	Purpose	Trials	Command
pilot	Validate the pipeline; power estimate	24	<code>ros2 run ubot_bench bench run pilot</code>
sim_core	RQ1/2/4: 4 stacks × 4 worlds × 20 seeds	640	<code>ros2 run ubot_bench bench run sim_core --parallel 3</code>
sim_core_tight	SLAM 0.1 m / 0.1 rad ablation, paired with sim_core	640	<code>ros2 run ubot_bench bench run sim_core_tight --parallel 3</code>
sim_degradation	RQ3: glass, walkers, degradation, low light	~1,120	<code>ros2 run ubot_bench bench run sim_degradation --parallel 3</code>
sim_to_real	RQ5: sim half of the real comparison	480	<code>ros2 run ubot_bench bench run sim_to_real --parallel 3</code>

Analysis:

```
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.report <experiment>
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.compare sim_core sim_core_tight
ros2 run ubot_bench bench list <experiment> -v          # progress per trial
```

Rules for campaigns

1. Run from a **clean, committed** tree. Each experiment records the commit and a dirty flag (`experiment.yaml`, `code_version`).
2. Close everything heavy first: interactive Gazebo, RViz, OBS. During the pilot the machine sat at load ≈ 50 on 12 cores (real-time factor ≈ 0.3). That is slow and it caused timeouts.
3. Run long jobs as user services so they survive closing terminals and sessions.
`scripts/run_experiment.sh` runs the experiment, then its analysis:

```
systemd-run --user --collect --unit=bench-sim-core --working-directory=$HOME/uni-bot \
  ~/uni-bot/src/ubot/ubot_bench/scripts/run_experiment.sh sim_core 3
systemctl --user status bench-sim-core          # running?
tail -f ~/uni-bot/bench_results/sim_core.log    # progress
systemctl --user stop bench-sim-core            # stop the orchestrator
```

If you stop it, kill the trial processes too (each trial is tagged GZ_PARTITION=bench_<trial>):

```
for d in /proc/[0-9]*; do { tr '\0' '\n' < $d/environ; } 2>/dev/null | grep -q
'^GZ_PARTITION=bench_' && kill -9 ${d#/proc/}; done
```

- Results are resumable. Re-running an experiment skips finished trials and retries infrastructure failures once. Task failures are data and are never retried.

5.1 Stopping, shutting down and continuing (do this yourself)

Experiments are **resumable**. Every finished trial has a `result.json` with a final status, and `bench run <experiment>` skips those. It re-runs only trials that are missing or were interrupted (no status, startup timeout, crash). A trial killed half-way is simply run again from its start; partial trials are never mixed into the results.

Before a shutdown

```
~/uni-bot/src/ubot/ubot_bench/scripts/stop_bench.sh
```

This stops every `bench-*` user service, kills all trial processes (Gazebo, Nav2, SLAM), and lists the trial folders left unfinished (they re-run automatically on resume). Shutting down without it is also safe, since the next run re-does interrupted trials. The script just makes the stop clean.

Continuing the pilot after a reboot. Step by step:

- Close heavy programs: interactive Gazebo, RViz, OBS. They slow the sim (real-time factor fell to ≈ 0.3 when they were open).
- Open a terminal and prepare the environment:

```
conda deactivate 2>/dev/null; cd ~/uni-bot
source /opt/ros/jazzy/setup.bash && source install/setup.bash
git status -s src/ubot      # should print nothing (clean tree). If not, commit or stash first
git log --oneline -1        # the commit the pilot will record
```

- Only if you changed code, worlds, missions or URDFs since the last build:

```
colcon build --symlink-install --cmake-args -DBUILD_TESTING=OFF && source
install/setup.bash
```

- Check what is done and what remains:

```
ros2 run ubot_bench bench list pilot      # counts per status
ros2 run ubot_bench bench list pilot -v   # every trial
```

- Start it as a background service, which keeps running if you close the terminal:

```
systemd-run --user --collect --unit=bench-pilot --working-directory=$HOME/uni-bot \
  ~/uni-bot/src/ubot/ubot_bench/scripts/run_experiment.sh pilot 1
```

If systemd says the unit name already exists, use another one, e.g. `bench-pilot-b`.

6. Watch progress (Ctrl-C only stops the viewer, not the run):

```
tail -f ~/uni-bot/bench_results/pilot.log # one line per finished trial
tail -f ~/uni-bot/bench_results/pilot/orchestrator.log
systemctl --user status bench-pilot # running or finished?
```

7. When it finishes, `run_experiment.sh` writes the analysis automatically to `~/uni-bot/bench_results/pilot/analysis/`. To redo the analysis by hand:

```
~/uni-bot/.venv-bench/bin/python -m ubot_bench.analysis.report pilot
```

8. **What to check in the pilot results** (analysis/):

- `trials.csv`: every trial `status = ok`, and every mapping trial has `map_saved` (see its `result.json`). Note any stuck.
- `success.csv`: house success should no longer collapse after one goal. A goal with `reset_after = True` in `goals.csv` was failed and reset; a long run of resets in one world means its goals or Nav2 settings need another look.
- `power.csv`: `n_required ≤ 20` for `slam_ate_rmse`, `map_f1`, `spl` and `success_rate` confirms `N = 20` seeds for `sim_core`. If not, raise `seeds` in that experiment file and record the change in `PROTOCOL.md` section 9.3.
- `figures/`: the figures exist and look sensible.

9. Then the campaigns, in this order (each the same way, with its own unit name): `sim_core` (parallel 3), `sim_core_tight`, `sim_degradation`, `sim_to_real`. Example:

```
systemd-run --user --collect --unit=bench-sim-core --working-directory=$HOME/uni-bot \
~/uni-bot/src/ubot/ubot_bench/scripts/run_experiment.sh sim_core 3
```

With an idle machine, an arena pair takes ~5 min and a house pair ~25 min, run one at a time. Use `parallel 3` only if the machine is otherwise idle; watch `uptime` (load should stay below the core count, 12).

Expected pilot duration from the stop point: 22 trials, about 2¼ hours on an idle machine (the last complete pilot took 2 h 17 min for 24 trials).

6. Work log (chronological)

6.1 29 Sep 2026: workspace moved to the latest branch

- **Problem.** `~/uni-bot/src/ubot` was an old, non-git copy.
- **Done.**
 - `~/uni-bot` became a checkout of `Unipod-Robotics/uni-bot`, branch `imu-bno085-ekf` (the newest; it contains `main`, `mono-nav`, `inspection-robot-description` and `hardware-interface`).
 - The old `src/ubot` and `src/MS200_ros` moved to `~/uni-bot-backup-2026-09-29/`. 17 files there were edited but never pushed (old box-model URDF, nav2/SLAM tuning).
 - Missing drivers imported: `bno08x_ros2_driver`, `topic_based_ros2_control`.
 - Clean build of 11 packages, with `-DBUILD_TESTING=OFF` (section 2.3).

6.2 29 Sep 2026: "the robot slides in Gazebo"; IMU axes stuck at the origin

- **Finding.** Headless ground-truth tests found 0.0 cm sideways drift in Gazebo, so the slide was in the estimated pose shown in RViz.

- **IMU axes at the map origin.** The RViz Imu display listened to `/bno055/imu`, which nothing publishes. Now every consumer uses `/imu`: the sim bridge remap was removed in `sim.launch.py` and `sim_mono.launch.py`, and `slam.rviz` and `sim_ekf.yaml` were updated.
- **Sim EKF.** It fused absolute x (not y) and absolute yaw from the wheels; wheel yaw under-reads turns by $\sim 4\%$. It is now velocities only (wheel vx and yaw rate, IMU yaw rate), the same as `real_ekf.yaml`.
- **Sim gyro.** It used MPU-9250 noise, with a random bias up to $\sim 0.3^\circ/\text{s}$. It now uses the BNO085 value measured in the E2 still test ($\sigma 0.0019 \text{ rad/s}$) with no static bias.
- **Result.** End-of-run heading error dropped from 3.2° to 0.1° on a 2 m / 90° / 2 m drive.
- **Commits** `cc1502c` and `4940ed9` (`slam_toolbox scan_topic: /scan`), pushed to `imu-bno085-ekf`.
- **Report.** `~/uni-bot/src/ubot_sim_imu_ekf_report.pdf`.

6.3 30 Sep 2026: decisions for the paper

Decision	Choice
Venue	IROS 2027 (deadline 1 Mar 2027). ICRA 2027 (16 Sep 2026) had passed
Real-robot ground truth	Overhead camera + fiducials
Hardware	Budget LiDARs (MS200, LD06) and OAK-D Lite. No reference-grade LiDAR , so REF is simulation-only and sim-to-real predictivity becomes RQ5
Scope of the build	Protocol plus full simulation benchmark; real-robot parts specified, built later
1 Oct: SLAM thresholds	Keep <code>slam_toolbox</code> defaults (0.5 m / 0.5 rad) and add the tight 0.1 m / 0.1 rad run (<code>sim_core_tight</code>)
1 Oct: RPLIDAR A1	Not used; removed from profiles, experiments, figures and protocol

6.4 30 Sep to 1 Oct 2026: benchmark built (commits `dc8cfbd`, `0fd141e`, and section 7)

Worlds (`ubot_worlds`)

- The worlds on the suggested list are Gazebo Classic. The house, bookstore and warehouse were taken from their community Harmonic ports, pinned in `turtlebot-maze/tb_worlds`; the hospital has no port. `import_aws_world.py` regenerates them:
 - our shared systems and lighting replace the originals;
 - the roof is dropped;
 - furniture is forced static. Many AWS models are non-static mesh bodies, which had put the house at $0.03\times$ real time;
 - ceiling lights become switchable for the low-light condition.
- `arena_5x5` is new, and it is the physical arena's build spec.
- Conditions are xacro args: glass panes (collision, invisible to LiDAR/depth via `visibility_flags`), walkers (actors), low light. Their placements live in `missions/<world>.yaml`.
- **Physics step 2 ms** in every world (the bookstore ran at $0.35\times$ real time at 1 ms).
- The worlds do **not** load the Sensors system: the robot URDF does, and loading it twice crashed Ogre.

Sensors (`ubot_description`)

- `config/sensor_profiles.yaml` holds the datasheet values for REF (UST-10LX), MS200, LD06 and OAK-D Lite. The MS200 manual gives 4,500 points/s (450 beams at 10 Hz); the "40,000

points/s" figure found online is the MS200k.

- The URDF builds the LiDAR from the profile (`sensor_profile:=`). `bench:=true` publishes a clean `/scan_raw` for `scan_model` to add datasheet noise; otherwise Gazebo adds constant noise on `/scan`.
- The bench bumper is a contact sensor on all chassis collisions. Gazebo publishes it on the scoped default topic, not the URDF `<topic>`.
- Wheel friction direction set to the axle (`fdir1 0 0 1`, `mu1` sideways 0.5, `mu2` rolling 1.0).
- In sim the rear wheels are driven explicitly (command interfaces; the `diff_drive` drives all 4) instead of mimic joints, which matches the ESP32 mirroring on the real robot.

Harness (`ubot_bench`)

- `scan_model` (noise), `gt_publisher` (ground truth and collision episodes), `gt_map` (ground-truth maps), `make_missions`, `mapping_runner`, `mission_runner`, `bench` (orchestrator) and `analysis/` (metrics, stats, figures, report, compare).
- `slam_toolbox` is started through its own launch file because it is a lifecycle node on Jazzy.
- OAK-D depth → scan reuses `ubot_mono_nav/depth_to_scan`, with its measured mount (TF off, because `camera_depth_frame` is not an optical frame). A centre-row `depthimage_to_laserscan` would see the floor at ~2.1 m because of the 4.09° tilt.

Findings and fixes during the build

#	Finding	Fix
1	Headless Gazebo used software rendering	NVIDIA EGL vendor variable (section 2.5)
2	Bookstore storefront is open at the 0.36 m scan plane (sill below), so goals were generated outside the building	Three-slice ground-truth scanner (0.08 / 0.22 / 0.36 m): scan-plane map plus traversability map
3	Warehouse bounds were guessed from a skewed, drift-built map	Bounds set just inside the measured walls ($x \pm 6.85$, $y \pm 10.33$ m)
4	In-place turns pivot about the front axle (base moves 0.25 m per 180°)	Not a bug. Coulomb friction makes the more-loaded axle the pivot (CoM $x = +0.013$ m); a 16 mm rearward CoM flips it to the rear axle. Collision shape, friction direction, wheel order, mimic and torque limits had no effect. Kept, documented, and measured on the real robot; position-only success added
5	Mapping route zig-zag (47°/m) made the follower crawl	Route straightening
6	The ground-truth follower hit a box	Route clearance 0.30 / 0.35 m, lookahead 0.30 m, <code>stuck</code> detection
7	Regenerated missions were not installed, so trials mixed old and new routes	Stale-install guard in <code>bench run</code>
8	Lifecycle activation timed out under load, so a trial hung	Bounded Nav2 startup wait; retried once
9	Nav2 behaviour-tree server timeout of 20 ms aborted goals under load	1000 ms (plus <code>wait_for_service_timeout</code> 5000 ms)
10	Background jobs died with the editor session	Long jobs run as user systemd services (section 5)

Each experiment that ran before a fix was set aside in `bench_results/_invalid_*` or `_pilot_arena_nav_lowtimeout` and re-run. Those results are not used.

Figures palette. The figures use categorical slots 1–4 of the reference palette for REF, MS200, LD06 and OAK-D, in that order (`#2a78d6`, `#eb6834`, `#1baf7a`, `#eda100`). The validator passes (worst adjacent CVD ΔE 9.1, normal-vision ΔE 22.9). Two colours are below 3:1 contrast, so every figure also carries visible stack labels.

6.5 1 Oct 2026: wheel-separation calibration; pilot restarted

- **Finding.** In-place turns overshoot by 4.5 % (`diag separation: true/commanded` = 1.045 at 0.3, 0.6 and 1.0 rad/s). The sim controller's `wheel_separation` 0.264204 was a leftover from an older robot model; its comment claimed a Gazebo check of 1.001 that no longer held after the chassis, wheel and friction changes. A ratio that is identical at every rate means a wrong constant, not slip.
- **Fix.** `ubot_bringup/config/sim_ubot_controllers.yaml` `wheel_separation` = 0.264204 / 1.045 = **0.2528 m**, with the comment rewritten.
- **Verification.** `diag separation` gives 1.000 at all three rates, `diag turn` gives 89.9° for 90°, and `diag yawrate` wheel odometry is within ~0.5 % of truth over the turn and settle.
- **Pilot.** The running pilot (calibrated with the old value) was stopped. Its results moved to `bench_results/_invalid_pilot_sep0264/`. The pilot was restarted from scratch at the commit that contains this fix.

6.6 1 Oct 2026: missing maps and cascading goal failures; pilot restarted

- **Missing maps.** 4 of 12 mapping trials ended `ok` but with no `map.yaml`; their navigation trials were skipped. The log showed `map_saver: Failed to spin map subscription`. Fix: the runner saves `/map` itself (7.8). Checked on an arena trial: map F1 1.00 and ATE 7.0 cm, as before.
- **Cascades.** House navigation succeeded only 25 % (REF) and 31 % (MS200), even with REF's localisation at 7.6 cm. Goal 3 at (6.78, 0.88) sat among dining-chair legs. The robot wedged there: "Failed to make progress" 69 times, and backup and spin aborted with "Collision Ahead". Every later goal then failed without the robot moving (0.00 m driven). Approved fix: goal clearance 0.60 m with ≥ 0.35 m passages (7.7), and independent goals with a reset after a failure (7.8).
- **Provenance.** `experiment.yaml` now keeps a `runs` list with the commit of every invocation.

6.7 Pilot status (stopped for a shutdown, 1 Oct 2026 08:50)

- The pilot was restarted from scratch after the 6.6 fixes (commit `77e6fd4`, `clean`) at 08:43.
- It was **stopped at 08:50** for a system shutdown, with `scripts/stop_bench.sh` added on that occasion. Done: arena REF seed 1 mapping (127 s) and navigation (170 s), 5 of 6 goals. Goal 2 showed a false success: Nav2 reported success, but ground truth was 0.26 m off. It was reset, and goals 3–6 then succeeded, so the independent-goal reset works.
- The interrupted trial (arena REF seed 2 mapping) was deleted. 22 trials remain. Continue with section 5.1. Log: `~/uni-bot/bench_results/pilot.log`; trial log: `bench_results/pilot/orchestrator.log`; results: `bench_results/pilot/analysis/`.
- Earlier, superseded pilot runs are kept for reference only, in `bench_results/_invalid_*` and `_pilot_arena_nav_lowtimeout`.

7. How everything was built from scratch

Sections 2–5 rebuild the setup from the repository. This section records **how each piece was made in the first place**: the commands, the reasoning behind every number, the algorithms, and the

check that confirmed each result. Someone with a bare ROS 2 Jazzy + Gazebo Harmonic install and this section could recreate the work without the repository. Subsections follow the order in which things were built.

7.1 Method used throughout

Every change was made in the same order: **measure** → **hypothesise** → **change one thing** → **measure again**.

- Ground truth always came from Gazebo itself: the model pose on `/world/<world>/pose/info`, read with the `gz-transport` Python bindings. It never came from the robot's own estimate.
- Tests ran headless in an isolated `ROS_DOMAIN_ID` and `GZ_PARTITION`, so they never touched a sim the user had open.
- The checks that produced the findings below are packaged as `ros2 run ubot_bench diag <test>`. Section 7.14 lists them with the values measured on 1 Oct 2026.
- **Important:** `ubot_worlds` and `ubot_description` install *copies* of their files, so every edit there needs a rebuild before it takes effect. Forgetting this mixed old and new routes in one pilot (finding #7). `bench run now` guards against it.

7.2 Workspace migration (29 Sep)

```
git ls-remote --heads <span class="url">https://github.com/Unipod-Robotics/uni-bot</span>
# list branches
git clone --filter=blob:none --no-checkout <span class="url">https://github.com/Unipod-
Robotics/uni-bot</span> /tmp/probe
git -C /tmp/probe for-each-ref --sort=-committerdate refs/remotes \
    --format='% (committerdate:short) % (refname:short) % (subject)'          # newest branch
git -C /tmp/probe merge-base --is-ancestor origin/main origin/imu-bno085-ekf # contains main?
```

- `imu-bno085-ekf` was newest (27 Sep) and contained `main`, `mono-nav`, `feat/inspection-robot-description` and `feat/hardware-interface`. On that branch the repo root is the workspace (it tracks `src/ubot` and `src/MS200_ros`).
- Before replacing anything, each local file was hashed (`git hash-object`) and checked against every blob in the remote history. 17 files had never been pushed, so the old tree was moved, not deleted:

```
mkdir ~/uni-bot-backup-2026-09-29 && mv ~/uni-bot/src/ubot ~/uni-bot/src/MS200_ros ~/uni-
bot-backup-2026-09-29/
cd ~/uni-bot && git init && git remote add origin <span
class="url">https://github.com/Unipod-Robotics/uni-bot.git</span>
git fetch origin && git checkout -b imu-bno085-ekf --track origin/imu-bno085-ekf
vcs import --skip-existing src < src/ubot/ubot.repos      # skip-existing keeps locally-
edited drivers
```

- `rosdep check` then showed two missing apt packages (section 2.1). The first build failed on `topic_based_ros2_control` (`ros_testing` not found), hence `-DBUILD_TESTING=OFF`.

7.3 IMU display and EKF investigation (29 Sep)

Is the robot really sliding? Drive it straight on ground truth.

- Result: 3.011 m forward, 0.0 cm sideways (`diag straight`).
- The same held through the full `sim.launch.py` stack, at 1.0 m/s, and after a turn.
- So Gazebo was fine, and the "sliding" was the *estimate* drawn in RViz.

IMU axes at the origin. `slam.rviz` listened to `/bno055/imu`; `ros2 topic list` showed it does not exist. The `rviz_imu_plugin` draws its axes at the fixed-frame origin until a message arrives. Fixed by using one topic name, `/imu`, everywhere (bridge, EKF, RViz).

EKF. `sim_ekf.yaml` fused wheel `x` (not `y`) and wheel `yaw` as absolute poses.

- Measured over a 2 m / 90° / 2 m drive: heading error 3.2°.
- The fix copies `real_ekf.yaml`'s velocity-only matrix: wheel `vx` and `vyaw`, IMU `vyaw`.

The sim gyro. With the new matrix, heading error rose to 8.7°. `diag yawrate` showed:

- the sim IMU drifted 0.86° per 5 s at rest (0.17°/s bias);
- the IMU's covariance was 370× smaller than the wheels', so the EKF trusted it almost entirely.

The noise block was labelled "MPU 9250" (σ 0.009, bias σ 0.005 rad/s). The fix sets the gyro σ to the BNO085's measured 0.0019 rad/s from the E2 still test ($\sqrt{3.7e-6}$) and removes the static bias. Heading error fell to 0.1°.

7.4 Choosing and porting the worlds (30 Sep)

1. **Survey.** All twelve worlds on the automaticaddison list are Gazebo Classic. A search found Harmonic ports for the AWS small house, small warehouse and bookstore, in unmerged PRs on the archived AWS repos. The hospital had none. The ports are packaged together in `zp78-ship-it/turtlebot-maze/tb_worlds`.
2. **Assets, pinned.** A sparse clone takes only `tb_worlds/models` at commit 30424d5 (`scripts/fetch_models.sh: git fetch --depth 1 --filter=blob:none origin <commit> with a sparse-checkout file`). The ~140 MB of meshes stay out of git.
3. **Import** (`scripts/import_aws_world.py`), applied to the port's `*.sdf.xacro`:
 - strip comments (the warehouse file had a broken one that left the roof in);
 - keep every top-level `<model>` except the roof and ground, and every `<light>` except `sun`;
 - force furniture static with `<include><static>true</static>`. The outer `<static>` does not reach an included model, and ~40 % of the AWS models are non-static mesh bodies. That alone put the house at **0.03× real time**; static furniture brought it to 0.47×;
 - wrap the indoor lights so the low-light condition can switch them off;
 - wrap everything in our header: shared systems, lighting, ground, condition geometry.
4. **Shared systems** (`worlds/include/ubot_world_common.xacro`): Physics, UserCommands, SceneBroadcaster (always on: it is the ground-truth source), Imu and Contact.
 - **Not Sensors.** The robot URDF already loads it, and a second instance created a duplicate Ogre scene and crashed (A material datablock ... already exists).
5. **Physics step.** Real-time factors measured with `gz topic -e -t /stats` on each world alone at 1, 2 and 4 ms. At 1 ms the bookstore ran at 0.36×; at 2 ms every world ran at $\geq 0.7\times$ with other load present. **2 ms is used everywhere**, so dynamics never differ between worlds.
6. **Headless rendering.** The logs showed `libEGL ... driver (null)` (software rendering). Fixed with `__EGL_VENDOR_LIBRARY_FILENAMES` pointing at NVIDIA's EGL vendor file.
7. **Arena.** Designed around the robot's scan plane:
 - The LiDAR height comes from the URDF TF chain (`xacro` → walk the joints): `lidar_link z = 0.3627 m`, `camera_link z = 0.152 m`.
 - So walls and obstacles are 0.60 m tall, enough for every 2D sensor to see them.
 - The obstacles create a narrow passage (P1), a near-centre cylinder (C1) and boxes of different sizes.

- Floor marks at $(\pm 2, \pm 2)$ serve the overhead-camera calibration.

8. Conditions.

- *Glass*: a pane with full collision whose visual has `visibility_flags = 2`, while the robot's LiDAR and depth camera use `visibility_mask 0xFFFFFFFF`. They see through it, and the robot still hits it. Checked with `diag glass` (beam reads 5.21 m, the pane is at 4.87 m).
- *Walkers*: Gazebo actors with a scripted `walk` trajectory. Waypoint times are segment length / speed, emitted by a recursive xacro macro. Actors are rendered (sensors see them) but have no collision.
- *Low light*: ambient and sun $\times 0.1$, indoor lights off.
- Placements live in `missions/<world>.yaml` and are read by xacro (`xacro.load_yaml`). One xacro trap: `{}` inside `${...}` ends the expression early, so the code uses `dict()`.

9. **Launch** (`launch/gz_world.launch.py`): it renders the xacro for (world, condition, physics) into a temp `.sdf`, starts `gz sim`, and exposes `spawn_pose()` from the mission file. Gazebo addresses worlds by name, and every world's `<world name>` equals its file name.

7.5 Sensor models (30 Sep)

1. **Datasheets**. One per sensor. The MS200 manual's specification table is an image; it was rendered with `pypdfium2` and read directly. It says 4,500 points/s, 5–15 Hz, 0.8° at 10 Hz, $\sigma \leq 4$ mm below 2 m and ≤ 15 mm from 2–12 m, and accuracy $\pm 10 / \pm 20$ mm. A web summary had claimed 40,000 points/s, which turned out to be the MS200k.
2. **Derivations** (all in `sensor_profiles.yaml` comments):
 - Beams per scan = points per second $\div$ scan rate. MS200 and LD06: $4500 / 10 = 450$. REF (UST-10LX): 0.25° over 270° gives 1081 beams.
 - Gazebo places beams on both endpoints, so a 360° sensor would put two beams on the same bearing. Angles are therefore $\pm(\pi - \pi/n)$, giving spacing $2\pi/n$ exactly.
 - Where a datasheet gives only an accuracy bound A , $\sigma = A/2$ (A read as $\sim 2\sigma$). The REF ± 40 mm gives σ 20 mm; the LD06 ± 45 mm gives 22.5 mm.
 - The OAK-D Lite stereo error grows as z^2 . With $\sigma = k \cdot z^2$ and $2\sigma = 2\%$ at 4 m: $k = 0.02 \cdot 4 / (2 \cdot 16) = 0.0025$. Check at 7 m: $2\sigma = 2 \cdot 0.0025 \cdot 49 = 0.245$ m = 3.5 %, inside the " $<4\%$ at 4–7 m" spec.
3. **URDF**. `ubot_gazebo.urdf.xacro` loads the profile with `xacro.load_yaml(...)[sensor_profile]` and shapes the `gpu_lidar` (rate, samples, angles, range).
 - xacro passes `bench:=false` as the *string* "false", which Python treats as true; it is normalised with `str(bench).lower()` in `('true', '1')`.
 - A depth profile omits the LiDAR entirely.
4. **Noise** (`scan_model` node): $\hat{r} = r + b(r) + N(0, \sigma(r)^2)$.
 - b is drawn once per run per band from $U(-B, B)$; returns pushed outside the sensor's range become "no return".
 - The C3 degradation adds 20 % dropout, 2 % spurious short returns, and $\sigma \times 3$.
 - It publishes *reliable*: `slam_toolbox` subscribes reliable, and a best-effort publisher would not match it.
5. **Depth $\rightarrow$ scan**. The OAK-D sits at 0.152 m, tilted 4.09° down, so its centre row meets the floor at $0.152 / \tan 4.09^\circ \approx 2.1$ m. `depthimage_to_laserscan` would report the floor as a wall. So `ubot_mono_nav/depth_to_scan` projects every pixel through the camera pose and keeps points 0.05–0.60 m high.
 - TF is off because `camera_depth_frame` is not an optical frame; the node's fallback mount equals the URDF pose.

6. **Prices** are approximate (September 2026), flagged in the file, and must be replaced with dated quotes.

7.6 Ground truth (30 Sep)

- **Pose** (`gt_publisher`). It subscribes to `/world/<world>/pose/info` (`Pose_V`) via `gz-transport` and publishes `/ground_truth/odom`, stamped with the pose message's sim time (the same clock as `/clock`). It never publishes on TF, so ground truth cannot leak into the estimation stack. The `ubot` entry is `base_footprint`; this was checked by comparing link poses in the same message.
- **Collisions**.
 - The contact-sensor collision names come from `gz sdf -p` on the generated URDF: lumping renames `base_link` collisions to `base_footprint_fixed_joint_lump__base_link_collision[1,2]`.
 - The URDF `<topic>` is ignored for contact sensors. `gz topic -l` showed the real topic, `/world/<w>/model/ubot/link/base_footprint/sensor/bumper/contact`.
 - Raw contacts arrive at ~400 messages per second of contact. They are turned into **episodes**: a new one starts after ≥ 0.5 s without contact with that object.
 - Checked with `diag bump`: reversing into a wall gives 1 episode.
- **Maps** (`gt_map`).
 - *Scanner*. No robot is involved. A collision-free, gravity-free model with three noiseless LiDARs (2880 beams, 30 m range) at 0.08 / 0.22 / 0.3627 m is spawned with the `gz create service` and moved with `set_pose`.
 - *Freshness*. `gpu_lidar` leaves `LaserScan.world_pose` empty, so a scan counts as post-teleport only when its stamp is ≥ 3 frames (0.15 s) after the request.
 - *Integration*. Per slice, hit and pass counts along every beam, sampled at half-cell steps. A cell both passed through and hit in the same scan counts as a hit (grazing beam).
 - *Exploration*. A 0.5 m lattice (0.75 m in the warehouse). A node is eligible once it is known free with ≥ 0.30 m clearance, inside bounds, and connected to the spawn. Nodes are visited breadth-first; eligibility uses a lenient one-observation labelling.
 - *Final labels*. Occupied if hits ≥ 2 and hit ratio ≥ 0.25 ; free if passes ≥ 2 .
 - *Why three slices*. With one slice, the bookstore came out "open": its storefront is windows above a sill, and the scanner left the building. The scan-plane map (z36) judges map quality; the union of all three slices gives traversability.
 - *Bounds*. The warehouse's were first guessed from a drift-skewed shipped map, then **measured** on the ground-truth map (rows and columns with long occupied runs: walls at $x \pm 6.85$, $y \pm 10.33$) and set just inside them.

7.7 Missions (30 Sep to 1 Oct)

`make_missions` works on the traversability grid, with glass panes rasterised in as occupied so missions are identical in every condition.

- *Feasible*. Clearance ≥ 0.25 m, where 0.213 m is the chassis circumscribed radius ($\sqrt{(0.1785^2 + 0.116^2)}$).
- *Goals*. Geodesic farthest-point sampling (Dijkstra, 8-connected) from the spawn among cells with ≥ 0.60 m clearance that are reachable through passages with ≥ 0.35 m clearance (distances measured along those passages). Visited in sampling order, with headings from seed 1. These limits came from the pilot: at 0.45 m a house goal landed among dining-chair legs and the robot wedged there.
- *Route*.

- Farthest-point coverage points until the largest gap is below the spacing (1.2 / 3.0 / 3.0 / 5.0 m), toured greedily and closed back at the spawn, which gives a loop closure.
- Joined by Dijkstra with cost $1 + 0.15 / \text{clearance}$, which prefers the middle of passages, over cells with ≥ 0.30 m clearance.
- The grid path is then straightened: extend each straight segment forward until it would drop below 0.35 m clearance or stray more than 0.25 m from the grid path. A first version that jumped to the farthest visible point skipped whole loops (route 0 m) and was replaced.
- Resampled every 0.10 m.
- *Why those clearances.* In an in-place turn, the chassis pivots on the front axle (7.11). The rear corners then sweep a radius of $\sqrt{(0.1785 + 0.127)^2 + 0.116^2} \approx 0.33$ m. At 0.25 m clearance the pilot follower hit a box.
- *Preview.* A PNG of each world's goals and route is written for inspection.

7.8 Runners (30 Sep to 1 Oct)

- **Mapping.**
 - Pure pursuit on ground truth: 0.20 m/s, lookahead 0.30 m, rotate in place above 45° , speed halved when $|\omega| > 0.8$ rad/s. Commands go straight to `/diff_drive_controller/cmd_vel` as a `TwistStamped`.
 - Progress is the nearest route index within the next 2 m of arc, and never goes backwards.
 - No progress for 30 s of sim time marks the trial `stuck`.
 - Trajectories are TUM files. The SLAM pose is looked up as TF `map` $\rightarrow$ `base_footprint` at every 5th ground-truth stamp (tf2 interpolates); lookups are retried for 2 s while TF catches up.
 - At the end, `slam_toolbox`'s `save_map` and `serialize_map` services are called.
- **Navigation.**
 - Goals are converted to the SLAM map frame as $T_{\text{spawn}}^{-1} \cdot \text{goal}$, because `slam_toolbox`'s map origin is where mapping started.
 - AMCL's initial pose is (0, 0, 0). Nav2 is driven with `nav2_simple_commander`.
 - Per-goal budget: $\max(60 \text{ s}, 4 \times \text{straight distance} / 0.25 \text{ m/s} + 30 \text{ s})$.
 - Success is judged on ground truth (≤ 0.25 m and ≤ 0.30 rad), never on Nav2's verdict, so "false success" is measurable.
 - `waitUntilNav2Active` is wrapped in a 180 s bounded wait, because it can block forever when a lifecycle transition is lost under load.
 - **Independent goals.** After a failed goal (when more goals follow), the runner:
 1. teleports the robot to the goal's true pose (`gz /world/<w>/set_pose, model ubot`);
 2. calls `setInitialPose` at the goal's map-frame pose;
 3. calls `clearAllCostmaps`;
 4. sets `reset_after` on that goal.

The teleport happens between two distance snapshots, so it never counts as a leg's path. Checked: the robot lands exactly on the requested pose ($-1.8, -1.8 \rightarrow 1.0, -0.5$).

- **Map saving.** The runner subscribes to `/map` (reliable, transient local) and writes the first message stamped after the robot stopped as trinary PGM/YAML (free ≤ 25 , occupied ≥ 65 , the same as `map_saver`). Then it calls `serialize_map`. `slam_toolbox`'s `save_map` service was dropped: its internal `map_saver` waits 2 s for `/map`, which `slam_toolbox` publishes every 5 s.

7.9 Nav2 and SLAM configuration (30 Sep to 1 Oct)

Both are derived programmatically from the existing sim configs (yaml load → edit → dump with an explanatory header), so every change is listed:

- **DWB.** 0.25 m/s and 1.0 rad/s, the real robot's controller limits from `ubot_controllers.yaml`.
- **Costmaps.**
 - Footprint is the chassis rectangle $\pm 0.18 \times \pm 0.12$ m (from the URDF base boxes), replacing a 0.15 m circle.
 - Inflation is 0.45 m in both maps.
 - Marking and clearing ranges are 2.5 / 3.0 m, below every sensor's range, so the costmaps are identical across stacks.
- **AMCL.** Likelihood field, 60 beams, `laser_max_range` per stack, `set_initial_pose` false (the runner sets it).
- **Behaviour tree.** `default_server_timeout` 20 → 1000 ms and `wait_for_service_timeout` 1000 → 5000 ms. At 20 ms, the planner's goal acknowledgement timed out on a loaded machine and goals were aborted.
- **slam_toolbox.** Mapping mode with no prior map. It is a lifecycle node on Jazzy: started bare, it never subscribed to `/scan`, so it is launched through its own `online_async_launch.py`, which configures and activates it.
- **Profiles.** `max_laser_range` per stack. The `tight` profile sets `minimum_travel_distance` and `minimum_travel_heading` to 0.1 and AMCL `update_min_a` to 0.1. It is applied with `nav2_common.RewrittenYaml`; checked with `ros2 param get (0.1 / 0.1 / 0.1)`.

7.10 Orchestration (30 Sep to 1 Oct)

- **Isolation.** Each trial gets a unique `ROS_DOMAIN_ID` (120–219), a `GZ_PARTITION` of `bench_<trial>_<pid>`, and `ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST`. The launch runs in its own process group.
- **Cleanup.** At the end, SIGINT goes to the process group, then SIGKILL to every process whose `/proc/<pid>/environ` carries the trial's partition. Gazebo forks out of the group, so killing the group alone left orphans.
- **Resources.** Every 2 s, psutil samples CPU % and RSS of the tagged processes, grouped by role (SLAM, AMCL, EKF, scan chain, Nav2 servers, Gazebo).
- **Scheduling.** Trials run in pairs: each mapping trial is followed immediately by its navigation trial on the map it just saved.
- **Retries and failures.**
 - Infrastructure failures (startup timeout, crash, runner timeout) are retried once.
 - Task outcomes, including `stuck`, are data and are never retried.
 - Re-running an experiment skips finished trials.
- **Provenance.** `experiment.yaml` records the workspace commit, a dirty flag and the models commit. A stale `ubot_worlds` install (source ≠ installed, byte compare) aborts the run.
- **Long runs.** These run as `systemd-run --user services`. The editor's background tasks are killed after 30 minutes, and a `setsid ... & disown` started inside one died with it.

7.11 The pivot investigation: a worked example of the method (30 Sep)

- **Symptom.** SLAM error spiked at in-place turns, and odometry error oscillated with heading instead of growing.
- **Measurement** (`diag pivot`). A circle fit to the ground-truth path during a 360° spin gives radius 0.127 m, centre (+0.127, +0.005) m in the body frame, which is the front axle.

Hypothesis	Test	Result
Ground-truth point is not <code>base_footprint</code>	Link poses in the same <code>pose/info</code>	<code>ubot = base_footprint</code> ; rejected
Friction direction spins with the wheel	<code>fdir1</code> → axle, swap μ	Unchanged (0.254 m per 180°); kept anyway as correct modelling
Rear wheels free-rolling (mimic gives no torque)	Rear command interfaces, 4-wheel diff drive	Unchanged; kept (matches the ESP32)
Cylinder contact artefact	Sphere collisions	Unchanged; reverted
Infinitely stiff velocity servos	1 N·m effort limit	Unchanged; reverted
Solver ordering	Rear wheels declared first	Unchanged; reverted
Chassis touching the floor	Remove the under-box collision	Unchanged; reverted
Different engine	<code>physics:=bullet-featherstone</code>	Robot does not turn at all; DART kept
Coulomb load bias	CoM x +0.013 → -0.016 m	Pivot flips to the rear axle (-0.127); confirmed

- **Conclusion.** With ideal Coulomb friction, the two axles' sideways scrub forces must balance, so the more heavily loaded axle stops sliding and becomes the pivot. This is real physics for a skid-steer, so it was documented, not "fixed". It is measured on the real robot, and it motivated the position-only success metric.

7.12 Metrics and statistics (30 Sep)

- **Trajectories.**
 - evo 1.31: association ≤ 20 ms; ATE after Umeyama SE(3) alignment without scale, and after first-pose alignment; RPE over 1 m.
 - Checked against evo's own CLI (`evo_rpe tum gt.tum odom.tum --delta 1 --delta_unit m`): identical values.
 - A planted ± 0.1 – 0.3 s time shift showed timing is not a significant error source.
- **Maps.**
 - The SLAM map is placed in the world by the SE(2) fit of its own trajectory to ground truth.
 - Precision, recall and F1 use a 10 cm tolerance via distance transforms; IoU covers the free space.
 - Only cells observable from the route are counted: ray casts on the ground-truth map from every 5th route point.
- **SPL.** Shortest feasible path on the traversability map from the leg's *actual* start, using the same Dijkstra and 0.25 m clearance as the missions.
- **Statistics** (`stats.py`).
 - Bootstrap median CIs (10,000 resamples, fixed seed) and Wilson CIs.
 - Friedman test; Wilcoxon signed-rank paired by seed with Holm correction; Cliff's δ .
 - Cochran's Q and exact McNemar for success; MixedLM robustness check; paired power with a 0.955 ARE factor.
 - `compare.py` pairs two experiments (default vs tight).
- **Figures.**
 - Palette slots 1–4 run through the data-viz validator (`validate_palette.js`): all hard checks pass, and the contrast warning is handled by visible labels.

- Figures use one panel per world with seed points and a median bar, Wilson bars for success, a log-price Pareto with direct labels, and a single-hue degradation heatmap.

7.13 Documents

- **Literature.** Every reference was found by web search and its title, authors and venue were confirmed on the publisher or arXiv page. Where a page did not show the authors, a second search confirmed them. Nothing was cited from memory.
- **PDFs.** `scripts/protocol_pdf.py`: Markdown → HTML (python-markdown; GitHub-style lists converted to its blank-line / 4-space form) → `google-chrome --headless --print-to-pdf`. The rendered pages were inspected as images before release.

7.14 Diagnostics reference (`ros2 run ubot_bench diag <test>`)

Start a fresh simulation for each test: `ros2 launch ubot_bench bench_sim.launch.py world:=arena_5x5 stack:=MS200 phase:=mapping` (for `glass`, use `stack:=REF condition:=glass`).

Test	Finding it reproduces	Measured 1 Oct 2026
straight	Gazebo does not slide sideways	3.001 m forward, 0.0 cm lateral; 4 wheels at 15.385 rad/s
turn	Commanded vs achieved turn	89.9° for 90° (0.999); it was 95.1° (1.057) before the calibration
separation	Effective wheel separation	0.2528 m; true/commanded 1.000 at 0.3 / 0.6 / 1.0 rad/s (was 1.045 at 0.264204)
pivot	In-place turns pivot on the front axle	Radius 0.128 m, centre (+0.127, +0.005) m
tilt	Chassis level at rest	Roll 0.00°, pitch 0.00°
yawrate	BNO085 sim gyro unbiased; wheel odometry heading	IMU still +0.03°/5 s; turn + settle: truth 90.3°, IMU 90.3°, wheels 90.8° (before the calibration: wheels ~4 % short)
ekf	Velocity-only EKF keeps heading	Yaw error 0.1° (position error 18 cm = the pivot: $2 \times 0.127 \times \sin 45^\circ$)
bump	Collision episodes	1 episode, <code>wall_west</code>
glass	LiDAR sees through glass	Beam crossing the pane at 4.87 m reads 5.21 m

8. Commits

Commit	Branch	Summary
6ab133d	imu-bno085-ekf (base)	BNO085 covariances from the E2 still test (pre-existing)
cc1502c	imu-bno085-ekf (pushed)	Sim EKF velocities-only, BNO085 gyro noise, <code>/imu</code> everywhere
4940ed9	imu-bno085-ekf (pushed)	<code>slam_toolbox scan_topic: /scan</code>
dc8cfbd	feat/nav-benchmark	Benchmark: worlds, sensor models, harness, protocol
0fd141e	feat/nav-benchmark	Pilot fixes, all ground truth and missions, protocol v1.2
df5caaf	feat/nav-benchmark	Tight-SLAM ablation, RPLIDAR A1 removed, Sources section, this log
651fbd5	feat/nav-benchmark	Record commit hash in this log
d06570e	feat/nav-benchmark	<code>diag</code> tests for every finding; section 7 "How everything was built from scratch"

Commit	Branch	Summary
0906083	feat/nav-benchmark	Sim wheel_separation calibrated to 0.2528 m (diag separation); run_experiment.sh; pilot restarted
fd4fc2e	feat/nav-benchmark	Record the restarted pilot in this log
14d456a	feat/nav-benchmark	Map saved from /map; per-run code versions
77e6fd4	feat/nav-benchmark	Goal clearance 0.60 m / 0.35 m passages; independent goals (reset); PROTOCOL v1.5; pilot restarted
(latest; see git log)	feat/nav-benchmark	Pilot stopped for shutdown; stop_bench.sh; section 5.1 (how to continue)

feat/nav-benchmark is **not pushed**. Push it with `git push git@github.com:Unipod-Robotics/uni-bot.git feat/nav-benchmark`.

9. Where things are

Path	What
src/ubot/ubot_bench/docs/PROTOCOL.md / .pdf	Research protocol, including all sources
src/ubot/ubot_bench/docs/REPRODUCE.md / .pdf	This document
src/ubot/ubot_bench/	Harness code, experiments (experiments/*.yaml), Nav2/SLAM configs
src/ubot/ubot_bench/ubot_bench/diagnostics.py	diag tests that reproduce every finding (7.14)
src/ubot/ubot_bench/scripts/run_experiment.sh, stop_bench.sh	Start an experiment (+ analysis) as a service; stop everything safely (5.1)
src/ubot/ubot_worlds/	Worlds, missions (+ PNG previews), ground-truth maps, model fetch script
src/ubot/ubot_description/config/sensor_profiles.yaml	Sensor models (datasheet values)
~/uni-bot/bench_results/<experiment>/	Trial outputs and analysis/ (git-ignored)
~/uni-bot/.venv-bench/	Analysis venv (git-ignored)
~/uni-bot-backup-2026-09-29/	Pre-migration copy with 17 unpushed local edits
~/uni-bot/src/ubot_sim_imu_ekf_report.pdf	IMU/EKF investigation report (section 6.2)

10. Open items

- Not built yet for the real robot: the overhead-camera ground-truth node (AprilTag, ≥ 20 Hz, TUM output), the INA219 power logging, and scripted mapping drives (PROTOCOL.md section 7).
- Sensor prices are approximate. Replace them with dated quotes.
- The BNO085's residual gyro bias has not been measured, so the sim gyro has zero static bias.
- Measure the real robot's in-place pivot point with the ground-truth rig.
- Install `ros-jazzy-tf-transformations` and `python3-smbus` (needed by the real-robot bringup).
- `ubot_bringup/launch/real_robot.launch.py` still carries a commented-out RPLIDAR A1 block from before this work; it is unused.